

# RTS AI Analysis Service PoC

A reusable LLM-backed backend service for transformation impact analysis, test planning, transaction sample discovery, and defect triage.

|              |                                                                |
|--------------|----------------------------------------------------------------|
| Audience     | AI use-case / API key review team                              |
| Decision ask | Approve controlled LLM API key/token access for a bounded PoC. |
| Date         | April 27, 2026                                                 |

**Positioning statement** This request is not for an AI knowledge base, chatbot, generic RAG demo, or personal Copilot prompt. It is for a reusable backend AI analysis service that can be called, governed, audited, and reused across RTS transformation workflows.

Prepared as a concise approval brief: business value first, controlled scope, measurable pilot outcomes.

# 1. Executive Summary

## What we request

Approve controlled LLM API key/token access for the **RTS AI Analysis Service PoC**. The PoC will validate whether LLM-backed analysis can reduce effort and risk in message transformation work by operating as a reusable backend service, not as an individual user assistant.

## Why RTS needs LLM API access

RTS transformation work is difficult because the source material is structured for systems, not humans. Messages, fields, nodes, rules, lookups, helpers, and conditional branches form deep dependency chains. Business meaning is highly context-dependent, and accuracy requirements are strict because incorrect transformations can cause downstream rejection, business processing issues, remediation cost, and release or rollback risk.

## Why Copilot alone is not enough

Copilot is useful for individual development assistance. It cannot replace a backend service surface that is callable by pipelines, test workflows, monitoring/alerting, Chat/Copilot surfaces, existing internal services, and other agents. The RTS use case requires permission control, auditability, structured inputs and outputs, cost tracking, reusable prompts/workflows, and consistent access to RTS context.

## Expected value

The PoC targets measurable improvement in:

- 1 understanding complex message and rule behavior;
- 1 identifying impact surfaces and dependency paths;
- 1 preparing test scenarios and regression focus areas;
- 1 finding representative historical transaction samples;
- 1 accelerating defect triage and monitor alert investigation;
- 1 reducing repeated SME explanations across BA, development, testing, and operations.

## Safety boundary

The service will not make autonomous production decisions, write to production systems, approve rules, generate final business rules, trigger releases, or replace reviewers. Outputs are decision-support material for human-owned analysis, testing, investigation, and approval processes.

## 2. Problem Statement

RTS transformation analysis is high-friction because the data and logic are precise, abstract, and context-heavy.

### The messages are hard to read directly

Transformation messages are structured for machine processing. A human reader may see nested fields and technical names, but not the business significance, condition path, downstream expectation, or historical reason behind a value.

### Meaning depends on context

A field or node can mean different things depending on product, source system, target system, channel, transaction pattern, rule branch, lookup result, and downstream consumer behavior. The same technical symptom may have different business implications in different scopes.

### Dependencies are deep

One field, helper, lookup, or conditional branch may influence multiple target fields, nodes, validation outcomes, test cases, and downstream behaviors. Manual analysis can miss non-obvious paths, especially when a dependency crosses mapping rules, helper logic, and message structures.

### Accuracy pressure is strict

Transformation errors are not cosmetic. Incorrect output can cause downstream rejection, incorrect business processing, production incident investigation, remediation effort, release delay, rollback planning, or emergency fixes.

### Supporting work is expensive

Even when the transformation logic is known, the surrounding work remains costly: preparing tests, finding real historical transaction samples, explaining behavior to multiple roles, and investigating defects or monitor alerts all consume SME time.

**Practical consequence** RTS does not only need faster search. It needs a controlled analysis service that can assemble the right context, explain dependency paths, propose investigation steps, and produce reviewable outputs for several operational workflows.

### 3. Proposed Capability

The proposed capability is the **RTS AI Analysis Service**: a reusable backend service powered by LLM API access and scoped RTS context.

#### What the service does

The service helps teams turn structured transformation context into reviewable analysis:

- 1 explain message fields, nodes, rules, lookups, helpers, and branch behavior in human-readable form;
- 1 analyze potential impact surfaces from a field, rule, helper, lookup, error symptom, or planned change;
- 1 explain dependency paths across source fields, transformation rules, helper logic, lookups, and target outputs;
- 1 generate test planning support, including regression focus areas, boundary cases, negative cases, and coverage gaps;
- 1 assist historical transaction sample discovery by converting analysis conditions into search strategies;
- 1 triage defects or monitor alerts by proposing likely causes, related rules, and first investigation steps;
- 1 summarize the same analysis for BA, developer, tester, support, operations, or reviewer audiences.

#### What the service does not do

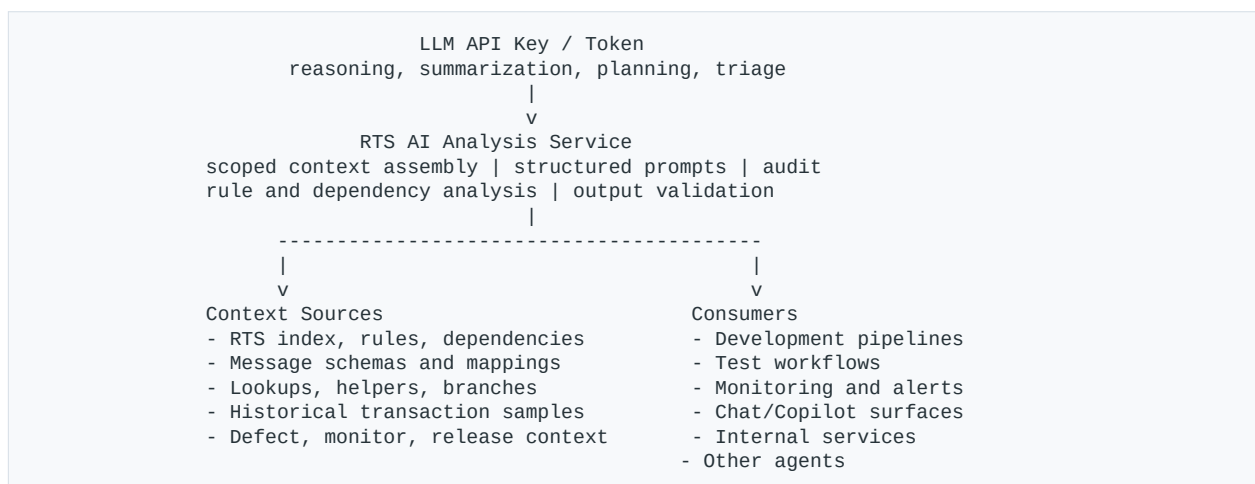
The service will not:

- 1 act as an AI knowledge base, chatbot, generic RAG app, or personal prompt wrapper;
- 1 write or modify production transformation rules;
- 1 approve business logic, tests, releases, or rollback decisions;
- 1 claim certainty where the available context is incomplete;
- 1 replace BA, developer, QA, support, operations, or reviewer responsibility.

#### Broader than change impact only

Change impact analysis is a high-value entry point, but the service is broader. It supports transformation workflows whenever a team needs to understand, analyze, validate, investigate, or communicate transformation behavior: before changes, during testing, after monitor alerts, while investigating defects, and during release or rollback risk review.

## 4. Minimal Architecture and Integration Surface



### Core dependency

The LLM API key is the core dependency that enables reasoning, summarization, test planning, sample search strategy generation, and defect triage over assembled RTS context. Without API access, RTS can still provide static indexes and search, but not a reusable analysis service surface.

### Service responsibilities

The RTS AI Analysis Service is responsible for:

- 1 assembling only the scoped context needed for a request;
- 1 applying service-owned prompt and output templates;
- 1 calling the LLM API under controlled credentials;
- 1 returning structured, reviewable outputs;
- 1 logging request source, context scope, model call metadata, cost, output version, and feedback where applicable.

### Integration surface

The same service capability can be reused from multiple entry points: development pipelines, test workflows, monitoring/alerting, Chat/Copilot surfaces, existing internal systems, and other agents. This reuse is the main reason API access is needed rather than relying on individual Copilot usage.

## 5. Reusable Use Cases and Benefits

### Message and rule understanding

Converts technical message structures and transformation rules into concise, role-appropriate explanations. Benefit: faster onboarding to complex context and fewer repeated SME explanations.

### Impact surface analysis

Starts from a field, node, rule, helper, lookup, planned change, or observed error and identifies likely affected targets, dependencies, scenarios, and uncertainty points. Benefit: fewer missed dependency paths and earlier risk visibility.

### Dependency path explanation

Shows how source data, rule logic, helper behavior, lookup outcomes, and branches contribute to a target value or downstream symptom. Benefit: reviewers and investigators can follow the reasoning path instead of relying on informal memory.

### Test planning support

Derives test focus areas, boundary conditions, negative cases, lookup miss/default branch cases, and regression checklist items from impact analysis. Benefit: less manual test preparation and more structured coverage review.

### Historical transaction sample discovery

Translates target scenarios into search strategies over historical real transaction data: field values, node presence, product types, condition combinations, edge cases, and missing coverage. Benefit: faster discovery of representative samples close to production behavior.

### Defect triage and monitor alert investigation

Uses alert details, rejection fields, message fragments, recent change context, and RTS dependencies to propose likely causes and first investigation steps. Benefit: shorter time to a useful hypothesis and less unstructured investigation effort.

### Release and rollback risk notes

Summarizes high-risk areas, coverage gaps, downstream attention points, and rollback considerations from the analysis. Benefit: clearer release conversations without delegating final decisions to AI.

### Cross-role summary

Produces consistent summaries for BA, development, QA, support, operations, and review audiences. Benefit: fewer repeated explanations and a more consistent shared understanding across roles.

## 6. Why LLM API, Not Only Copilot

### Where Copilot helps

Copilot is valuable for individual productivity:

- 1 code completion and local code explanation;
- 1 drafting tests or helper snippets;
- 1 ad hoc Q&A in IDE or chat surfaces;
- 1 quick assistance for a single developer session.

### Where Copilot is insufficient for this request

The RTS requirement is service-level, not individual-assistant-level. Copilot alone does not provide:

- 1 a stable backend API callable by pipelines, test workflows, monitor alerts, internal services, Chat/Copilot surfaces, and other agents;
- 1 centrally governed RTS context assembly;
- 1 consistent service prompts, structured outputs, and evaluation traces;
- 1 permission-controlled access to rules, dependencies, historical samples, defect context, and monitoring context;
- 1 audit logs for who called the service, what scope was used, what model was invoked, and what output was returned;
- 1 cost controls and reuse across teams and systems.

### Why API access is the right control point

An API-backed service allows the organization to place governance around a shared capability: credentials, permissions, logging, prompt templates, output contracts, cost limits, and evaluation metrics. This is safer and more reusable than distributing similar analysis through personal prompts.

**Key approval point** The LLM API key is not requested to make AI more conversational. It is requested to make RTS analysis callable, auditable, permission-controlled, reusable, and measurable as a backend service.

## 7. PoC Scope, Safety, and Review Controls

### Bounded PoC scope

Initial PoC materials should be sanitized, non-production, or otherwise controlled. If historical transactions, defect data, monitoring context, or release context are included, access should be explicitly scoped and limited to the minimum needed for evaluation.

### Controls to apply

- 1 Minimal permission for the API key and connected context sources.
- 1 No production writes and no automatic modification of transformation rules.
- 1 No autonomous approval of business rules, tests, releases, or rollback decisions.
- 1 Human-owned final judgment for analysis conclusions, test readiness, incident response, and release decisions.
- 1 Logging of caller, request type, context scope, model metadata, token/cost usage, output version, and feedback.
- 1 Baseline checks for sensitive-field leakage, over-broad retrieval, prompt injection, and inappropriate cross-scope context use.
- 1 Cost and rate limits suitable for a bounded PoC.

### Reviewable outputs

Outputs should be structured so reviewers can accept, reject, or challenge them:

- 1 answer summary;
- 1 relevant fields, rules, dependencies, or sample search criteria;
- 1 assumptions and uncertainty points;
- 1 suggested next steps;
- 1 explicit statement when evidence is insufficient.

### Out of scope

The PoC does not attempt to build a general AI platform, replace existing governance, automate production deployment, or make the LLM an authoritative source of business truth.



## 8. Evaluation Metrics and Final Ask

### PoC evaluation metrics

The review should measure whether the service produces practical, reviewable value:

- 1 analysis time for complex message/rule understanding;
- 1 dependency coverage compared with SME or reviewer baselines;
- 1 test preparation time from issue/change context to reviewable scenarios;
- 1 time to identify representative historical transaction sample candidates;
- 1 time from defect/alert input to first useful triage hypothesis;
- 1 reduction in repeated SME clarification across BA, development, testing, and operations;
- 1 adoption rate of generated outputs after human review;
- 1 successful integration through at least two surfaces, such as test workflow plus Chat/Copilot, or pipeline plus monitoring.

### Decision criteria after pilot

Continue or expand only if the PoC demonstrates:

- 1 outputs are useful enough for human reviewers to adopt or partially adopt;
- 1 safety controls keep context, permissions, logs, and costs within the approved boundary;
- 1 the service works as a reusable backend capability rather than a one-off prompt;
- 1 the benefits justify ongoing API usage and operational ownership.

### Final ask

Approve:

- 1 Controlled LLM API key/token access for the RTS AI Analysis Service PoC.
- 2 A bounded pilot scope using sanitized, non-production, or explicitly controlled RTS context.
- 3 Review of pilot results against the metrics above before any broader rollout.

**Approval summary** This is a controlled request to evaluate whether LLM API access can reduce RTS transformation analysis, testing, sample discovery, and triage effort through a reusable backend service. It does not request autonomous production authority, production writes, or replacement of human review.